



5.1 Evaluacija i zašto je manje više



5.1 Evaluacija i zašto je manje više

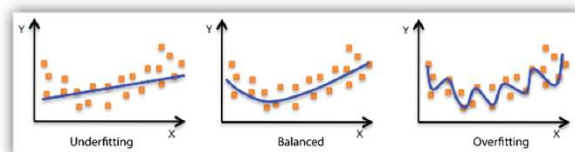
- Prvi korak: podijelite naše podatke u zasebne skupove podataka za obuku i testiranje
- Drugi korak: učite na podacima o obuci.
- Treći korak: procijenite podatke testiranja

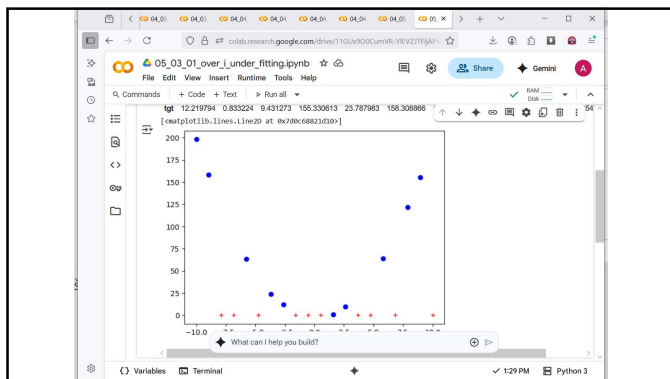
5.2 Terminologija za faze učenja

- Treniranje
- Validacija
- Testiranje

Skup podataka	Svrha	Koristi se kada?	Model ima pristup?	Tipične proporcije
Treniranje	Uči model na temelju uzoraka (podešavanje težina/parametara)	Tijekom procesa treniranja	☑ Da	50–80%
Validacija	Podešava hiperparametre, prati overfitting/underfitting	Tijekom treniranja (ali izvan učenja parametara)	☑ Da (ne izravno)	10–25%
Testiranje	Konačna provjera performansi modela na neviđenim podacima	Nakon što je model potpuno istreniran	☒ Ne	10–25%

5.3. Overfitting i underfitting





```
[9] import sklearn.linear_model as linear_model
sk_model = linear_model.LinearRegression()
sk_model.fit(train_ftr.reshape(-1, 1), train_tgt)
sk_preds = sk_model.predict(test_ftr.reshape(-1, 1))
sk_preds[1]
```

array([80.82419063, 78.81286676, 92.89213387])

What can I help you build?

5.3.1. Manipuliranje sa kompleksnosti



Google
colab

```
[9] import sklearn.linear_model as linear_model
sk_model = linear_model.LinearRegression()
sk_model.fit(train_ftr.reshape(-1, 1), train_tgt)
sk_preds = sk_model.predict(test_ftr.reshape(-1, 1))
sk_preds[1]
```

array([80.82419063, 78.81286676, 92.89213387])

```
[10] model_one = np.polyfit(train_ftr, train_tgt, 1)
preds_one = model_one(test_ftr)
print(preds_one[1])
```

[80.82419063 78.81286676 92.89213387]

What can I help you build?

```
[10] model_one = np.polyfit(train_ftr, train_tgt, 1)
preds_one = model_one(test_ftr)
print(preds_one[1])
```

[80.82419063 78.81286676 92.89213387]

```
from sklearn import metrics
# da li su predikcije iste?
print("Isto?", np.allclose(sk_preds, preds_one))
# and we can still use sklearn to evaluate it
mse = metrics.mean_squared_error
print("RMSE:", np.sqrt(mse(test_tgt, preds_one)))
```

Isto? True
RMSE: 67.275718703966

What can I help you build?

```
mse = metrics.mean_squared_error
print("RMSE:", np.sqrt(mse(test_tgt, preds_one)))
```

Isto? True
RMSE: 67.275718703966

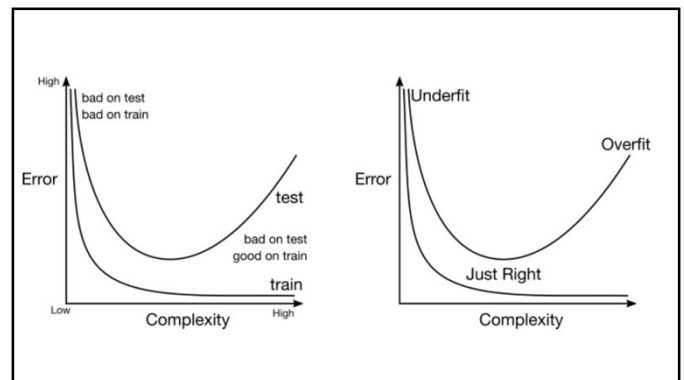
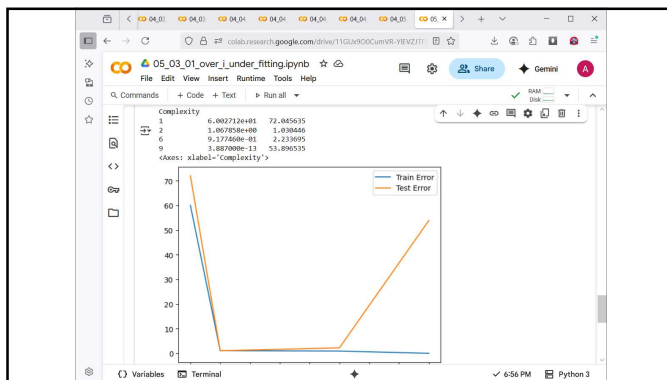
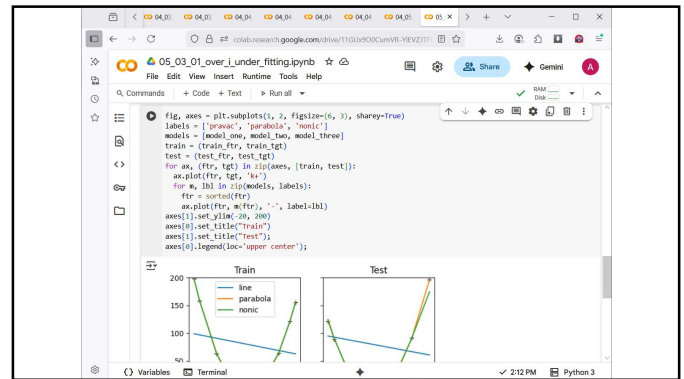
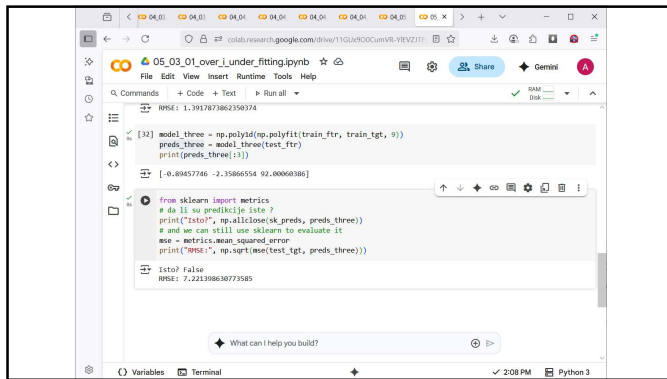
```
[13] model_one = np.polyfit(train_ftr, train_tgt, 2)
preds_one = model_one(test_ftr)
print(preds_one[1])
```

[-2.90731141 -3.81018098 91.83166063]

```
from sklearn import metrics
# da li su predikcije iste?
print("Isto?", np.allclose(sk_preds, preds_one))
# and we can still use sklearn to evaluate it
mse = metrics.mean_squared_error
print("RMSE:", np.sqrt(mse(test_tgt, preds_one)))
```

Isto? False
RMSE: 1.3917873862350374

What can I help you build?



5.4 Od pogrešaka do troškova

- Gubitak (Loss)
- Funkcija troška (Cost)
- Score (Rezultat ili funkcija bodovanja)

5.4.1 Gubitak (Loss)

$$\text{TrainingLoss}_{\text{Loss}}(m, D_{\text{train}}) = \sum_{x, y \in D_{\text{train}}} \text{Loss}(m(x), y)$$

$$\text{TrainingLoss}_{\text{SE}}(3\text{-NN}, D_{\text{train}}) = \sum_{x, y \in D_{\text{train}}} \text{SE}(3\text{-NN}(x), y) = \sum_{x, y \in D_{\text{train}}} (3\text{-NN}(x) - y)^2$$

Google
colab

```
def training_loss(loss, model, training_data):
    "total training loss on train data with model under loss"
    return sum(loss(model.predict(x.reshape(1, -1)), y)
               for x, y in training_data)

def squared_error(prediction, actual):
    "squared error on a single example"
    return (prediction - actual)**2
# koristimo je u odliku:
# moj gubitak tokom treniranja = training_loss(squared_error, model, training_data)
import numpy as np
from sklearn import neighbors, datasets

diabetes = datasets.load_diabetes()
knn = neighbors.KNeighborsRegressor(n_neighbors=3)
fit = knn.fit(diabetes.data, diabetes.target)
training_data = zip(diabetes.data, diabetes.target)
my_training_loss = training_loss(squared_error, knn, training_data)
print(my_training_loss)

[863792.33333333]
```

- mean_squared_error



```
from sklearn import metrics

mse = metrics.mean_squared_error(diabetes.target, knn.predict(diabetes.data))
print(mse*len(diabetes.data))

863792.33333333
```

5.4.2 Funkcija troška (Cost)

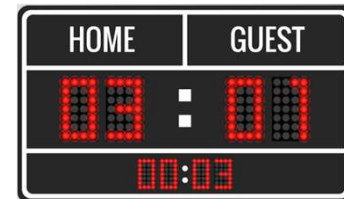
- trošak = gubitak + složenost

$$\text{Cost}(m, D_{\text{train}}, \text{Loss}, \lambda) = \text{TrainingLoss}_{\text{Loss}}(m, D_{\text{train}}) + \lambda \text{Complexity}(m)$$

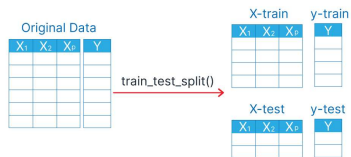


```
def complexity(model):
    return model.complexity
def cost(model, training_data, loss, _lambda):
    return training_loss(model, training_data) + _lambda * complexity(model)
```

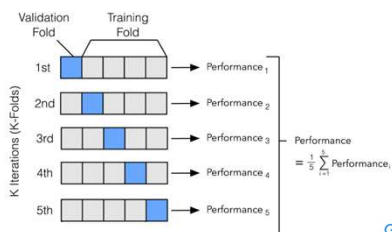
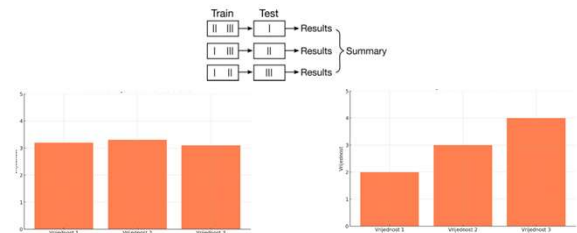
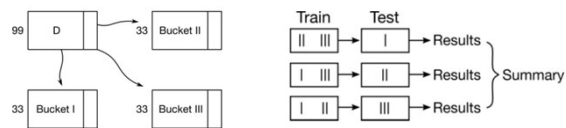
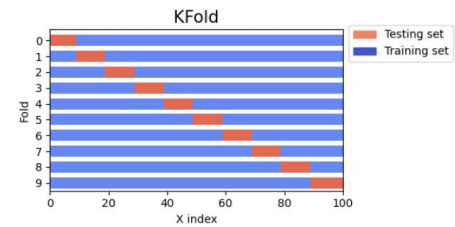
5.4.3 Score (Rezultat ili funkcija bodovanja)



5.5. (Re)uzorkovanje: iz manje napraviti više



5.5.1 Cross-Validation (Unakrsna provjera valjanosti)



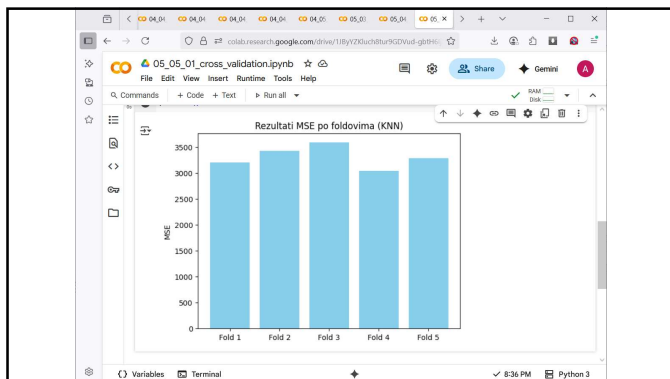
A screenshot of a Google Colab notebook titled '05_05_01_cross_validation.ipynb'. The code imports `sklearn` and `datasets`, loads the `diabetes` dataset, and performs K-fold cross-validation using `neighbors` and `metrics`. The output shows the mean cross-validated score and standard deviation.

```
from sklearn import datasets, neighbors, metrics, model_selection as skms
diabetes = datasets.load_diabetes()

# data, model, fit & cv-score
model = neighbors.KNeighborsRegressor(10)
scores = skms.cross_val_score(model,
                               diabetes.data,
                               diabetes.target,
                               cv=5,
                               scoring='neg_mean_squared_error')
# zadano za cross_val_score:
# cv=5 fold, no shuffle, stratified if classifier
# model.score zadano (regresor: r2, klasifikator: accuracy)
print(scores)
print(scores.mean())
print(scores.std())
```

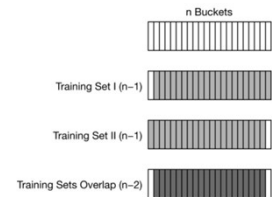
The output of the code is:

```
[-3200.7541573 -3426.43134831 -3587.942215989 -3839.49443182
 -3282.60159891]
-3188.644775872323
187.28724369817584
```



Koji k odabrati ?

- $k > k$ – više kombinacija i faza obuke i testiranja
- $2 < k < n$ – ekstremi ?



Koji k odabrati ?

- $2 < k < 10$ – manje vrijednosti ?

5.5.2 Stratifikacija

- **Stratifikacija** u strojnom učenju znači da se podaci dijele tako da omjeri klasa ostanu približno isti u svim podskupovima (npr. u treningu i testu, ili u svakom foldu kod cross-validationa)



```
from sklearn import datasets, neighbors, metrics, model_selection as skms

iris = datasets.load_iris()
model = neighbors.KNeighborsClassifier(10)
non_strat_kf = skms.KFold(5)
i1az1 = skms.cross_val_score(model,
                              iris.data,
                              iris.target,
                              cv=non_strat_kf)
print(i1az1)
```

[1. 1. 0.86666667 0.96666667 0.76666667]

```
from sklearn import datasets, neighbors, metrics, model_selection as skms

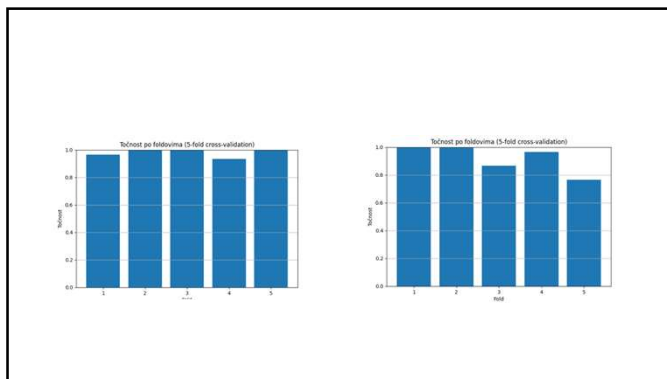
iris = datasets.load_iris()
model = neighbors.KNeighborsClassifier(10)
non_strat_kf = skms.KFold(5)
i1az1 = skms.cross_val_score(model,
                              iris.data,
                              iris.target,
                              cv=non_strat_kf)
print(i1az1)
```

[1. 1. 0.86666667 0.96666667 0.76666667]

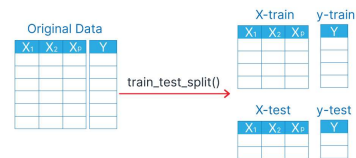
```
[4] iris = datasets.load_iris()
model = neighbors.KNeighborsClassifier(10)
i1az2 = skms.cross_val_score(model, iris.data, iris.target, cv=5)
print(i1az2)
```

[0.96666667 1. 0.93333333 1.]

```
[4] # bez strat!
import numpy
pet = np.array(['cat', 'dog', 'cat', 'dog', 'dog'])
```



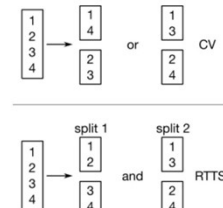
5.5.3. Ponovljeni train_test_split



Google
colab

```
from sklearn import datasets, metrics, model_selection as skms
from sklearn import linear_model
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd

linreg = linear_model.LinearRegression()
diabetes = datasets.load_diabetes()
scores = []
for i in range(10):
    tts = skms.train_test_split(diabetes.data,
                                diabetes.target,
                                test_size=.25)
    (diabetes_train_ftrs, diabetes_test_ftrs,
     diabetes_train_tgt, diabetes_test_tgt) = tts
    fit = linreg.fit(diabetes_train_ftrs, diabetes_train_tgt)
    preds = fit.predict(diabetes_test_ftrs)
    score = metrics.mean_squared_error(diabetes_test_tgt, preds)
    scores.append(score)
scores = pd.Series(np.sqrt(sorted(scores)))
df = pd.DataFrame({'RMSE': scores})
df.index.name = 'Iteration'
display(df.T)
```



5.5.4. Bolji način miješanja

- ShuffleSplit() !



Google
colab

```
from sklearn import linear_model, datasets, model_selection as skms
import pandas as pd
import numpy as np
import seaborn as sns

linreg = linear_model.LinearRegression()
diabetes = datasets.load_diabetes()
# n isy zadana cv argument
ss = skms.ShuffleSplit(test_size=.25, # default, 10 skupova
                        cv=ss,
                        scoring='neg_mean_squared_error')
scores = pd.Series(np.sqrt(-scores))
df = pd.DataFrame({'RMSE': scores})
df.index.name = 'Iteration'
display(df.describe().T)
ax = sns.heatmap(df['RMSE'], data=df)
ax.set_xlabel('Over-RepeatedTrain-Test Splits')
```

```

from sklearn.cross_validation import ShuffleSplit
diabetes = datasets.load_diabetes()
X = diabetes.data
y = diabetes.target

ss = ShuffleSplit(n_splits=10, random_state=0)
for train, test in ss.split(X, y):
    print("Train: %d, Test: %d" % (train, test))

```

array([16, 408, 432, 316, 3, 18, 355, 60, 398, 124])

```

from sklearn.cross_validation import ShuffleSplit
diabetes = datasets.load_diabetes()
X = diabetes.data
y = diabetes.target

ss = ShuffleSplit(n_splits=10, random_state=0)
for train, test in ss.split(X, y):
    print("Train: %d, Test: %d" % (train, test))

```

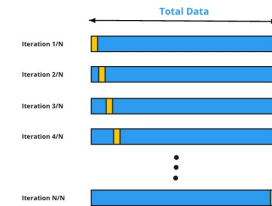
array([16, 408, 432, 316, 3, 18, 355, 60, 398, 124])

Kfold zadane vrijednosti!

- `skms.KFold(n_splits=3, shuffle=False, random_state=None)`
- `skms.KFold(n_splits=3, shuffle=True, random_state=None)`

5.5.5 Leave-One-Out Cross-Validation

LOOCV: Leave One Out Cross Validation



Google
colab

```

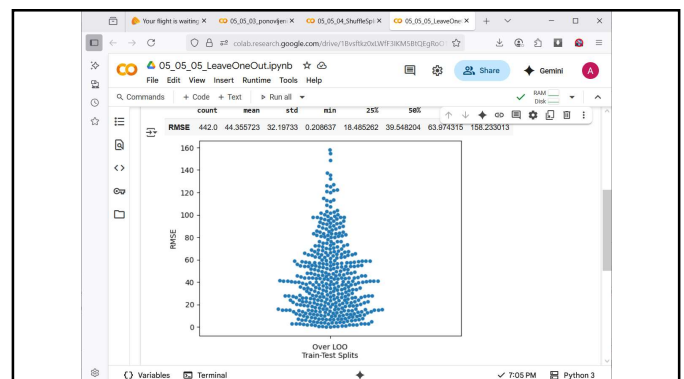
from sklearn import linear_model, datasets, model_selection as skms
import numpy as np
import pandas as pd
import seaborn as sns

linreg = linear_model.LinearRegression()
diabetes = datasets.load_diabetes()
loo = skms.LeaveOneOut()
scores = skms.cross_val_score(linreg,
                              diabetes.data, diabetes.target,
                              cv=loo,
                              scoring='neg_mean_squared_error')
scores = pd.Series(np.sqrt(-scores))
df = pd.DataFrame({'f1': scores})
df.index.name = 'f1'
display(df.describe())
ax = sns.scatterplot(y='f1', data=df)
ax.set_xlabel('Over 1000 train-test splits')

```

count mean std min 25% 50% 75% max

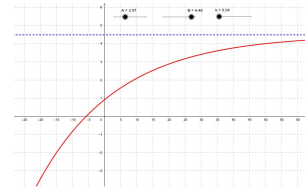
RMSE 442.0 44.355723 32.18733 0.208637 18.485262 39.548204 63.974315 158.233013



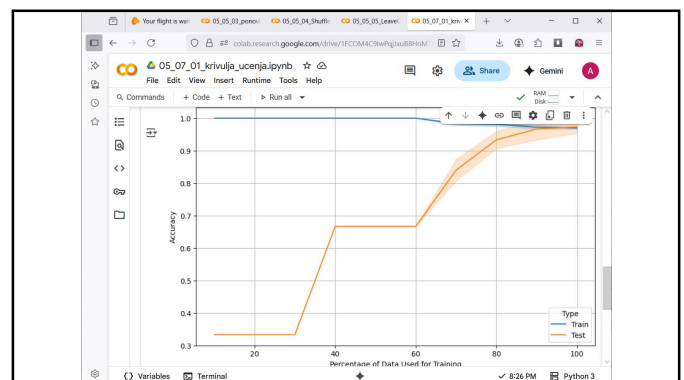
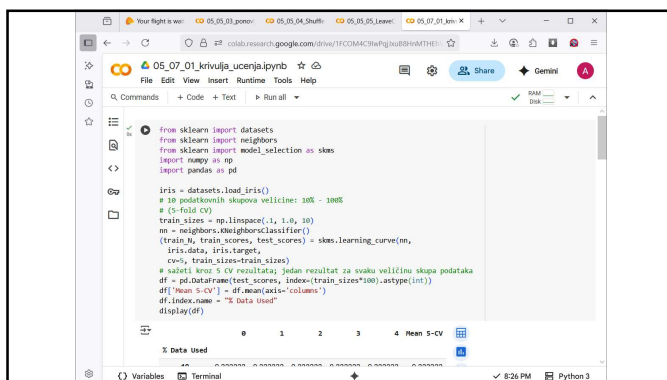
5.6 Raščlamba: Dekonstruiranje pogreške na pristranost i varijancu

- Varijanca podataka
 - Varijanca modela
 - Pristranost modela
- GREŠKA= Pristranost modela + Varijanca podataka + Varijanca modela

5.7.1 Krivulje učenja: Koliko podataka trebamo?



Google
colab



5.7.2 Krivulje složenosti

